Bagunceitor

Trabalho 1 · Programação de Baixo Nível · PUCRS

Estevam Cabral Pacheco

Repositório: https://github.com/estevamcabral/pbn-01

---

1. Contexto

O trabalho pede duas regras próprias para embaralhar uma imagem que já está carregada na memória. A primeira é a confusão, que altera o valor das cores. A segunda é a difusão, que altera a posição dos pixels. As duas funcionam de forma independente, e cada uma tem uma reversão que devolve a imagem original sem perder nenhum byte.

O programa foi escrito em C a partir do código base da disciplina, que usa a biblioteca stb_image para ler e gravar as imagens. Nos testes usei a foto `predio32.jpg`, de 1280 × 960 pixels.

2. A imagem na memória

Cada pixel é uma struct com três campos `unsigned char` (r, g e b), então ocupa 3 bytes. A imagem inteira fica num único vetor de pixels, guardado linha após linha.

```
typedef struct {
  unsigned char r, g, b;
} Pixel;
```

Com 1280 × 960 = 1.228.800 pixels, cada imagem ocupa 3.686.400 bytes, cerca de 3,5 MB.

Como o programa usa a memória

O programa trabalha com três vetores do mesmo tamanho:

vetor	alocado por	guarda
`in.pixels`	`stbi_load` (código base)	a imagem original
`out.pixels`	`malloc` (código base)	a imagem confundida
`recuperada`	`malloc` adicionado no trabalho	a imagem recuperada da confusão

A confusão lê a original e grava o resultado em `out`, sem alterar a original, que ainda é usada depois pela difusão. A reversão da confusão lê `out` e grava em `recuperada`. A difusão é a última etapa e trabalha dentro do próprio vetor da original: troca os pixels de lugar e depois desfaz as trocas, então não precisa de memória extra. No final, os três vetores são liberados com `free`.

```
int tam = in.width * in.height;
out.pixels = malloc(tam * sizeof(Pixel));
Pixel* recuperada = malloc(tam * sizeof(Pixel));
```

Ponteiros e aritmética de ponteiros

As funções recebem o endereço inicial de cada vetor, um `Pixel*`. Para chegar no pixel da posição `i`, o programa soma `i` a esse endereço: `origem + i` aponta para o pixel `i`. Como cada `Pixel` ocupa 3 bytes, somar 1 ao ponteiro avança 3 bytes na memória. O operador `->` acessa as cores do pixel apontado, e o operador `*` lê ou escreve o pixel inteiro naquele endereço.

```
Pixel* o = origem + i;
Pixel* d = destino + i;
d->r = o->g + i;
```

O tipo `unsigned char` também é importante para a confusão, porque só guarda valores de 0 a 255. Quando uma soma passa de 255, o valor dá a volta e recomeça do zero. Quando uma subtração fica negativa, ele dá a volta no sentido contrário. Por isso é possível somar e depois subtrair sem perder informação.

Primeiros 9 bytes (3 pixels) de cada imagem gerada pelo programa, em hexadecimal:

imagem	pixel 0	pixel 1	pixel 2
original	`A6 E4 FB`	`A6 E4 FB`	`A6 E4 FB`
confundida	`E4 FB A6`	`E5 FC A7`	`E6 FD A8`
recuperada (confusão)	`A6 E4 FB`	`A6 E4 FB`	`A6 E4 FB`
difundida	`A6 E4 FB`	`74 A7 9E`	`91 F0 FF`
recuperada (difusão)	`A6 E4 FB`	`A6 E4 FB`	`A6 E4 FB`

Na confundida dá para ver as cores girando e somando a posição (+0, +1, +2). Na difundida os valores são de pixels que vieram de outras posições. As duas recuperadas voltam aos mesmos bytes da original.

3. Confusão

Estratégia

Cada pixel passa por duas mudanças. As cores giram: o vermelho recebe o valor do verde, o verde recebe o do azul e o azul recebe o do vermelho. Depois, cada cor soma a posição `i` do pixel na imagem. Como a posição muda de pixel para pixel, cores iguais em lugares diferentes viram cores diferentes.

```
r' = g + i
g' = b + i
b' = r + i
```

Reversão

Para voltar, cada cor sai do lugar para onde foi e subtrai a mesma posição. A soma e a subtração do mesmo `i` se cancelam, e cada cor volta exatamente ao valor original.

```
r = b' − i
g = r' − i
b = g' − i
```

Código

```
void confundir(Pixel* origem, Pixel* destino, int tam) {
  for (int i = 0; i < tam; i++) {
    Pixel* o = origem + i;
    Pixel* d = destino + i;
    d->r = o->g + i;
    d->g = o->b + i;
    d->b = o->r + i;
  }
}

void desconfundir(Pixel* origem, Pixel* destino, int tam) {
  for (int i = 0; i < tam; i++) {
    Pixel* o = origem + i;
    Pixel* d = destino + i;
    d->r = o->b − i;
    d->g = o->r − i;
    d->b = o->g − i;
  }
}
```

Resultado

As cores mudam bastante, mas o contorno do prédio continua visível, porque nenhum pixel sai do lugar. A imagem recuperada foi comparada com a original byte a byte e é idêntica.

4. Difusão

Estratégia

Aqui as cores não mudam, só a posição. O programa percorre os pixels do primeiro ao último e troca cada pixel `i` com o pixel `j`, calculado pela fórmula abaixo. A multiplicação espalha pixels vizinhos para longe, e o resto da divisão mantém `j` dentro da imagem. No trabalho uso `numero = 10003`.

```
j = (i * numero) % tam
troca *(img + i) com *(img + j)
```

Reversão

A volta usa a mesma fórmula, só que percorrendo `i` do último pixel até o primeiro. O mesmo `i` sempre gera o mesmo `j`, e trocar os mesmos dois pixels de novo desfaz a troca. Assim, as trocas são desfeitas na ordem inversa. Como nenhum pixel é sobrescrito, a regra funciona com qualquer valor de `numero`.

`i` e `j` são `long long`, porque `i × 10003` passa do limite de um `int`.

Código

```
void difundir(Pixel* img, int tam, int numero) {
  for (long long i = 0; i < tam; i++) {
    long long j = (i * numero) % tam;
    Pixel* a = img + i;
    Pixel* b = img + j;
    Pixel t = *a;
    *a = *b;
    *b = t;
  }
}

void desdifundir(Pixel* img, int tam, int numero) {
  for (long long i = tam - 1; i >= 0; i--) {
    long long j = (i * numero) % tam;
    Pixel* a = img + i;
    Pixel* b = img + j;
    Pixel t = *a;
    *a = *b;
    *b = t;
  }
}
```

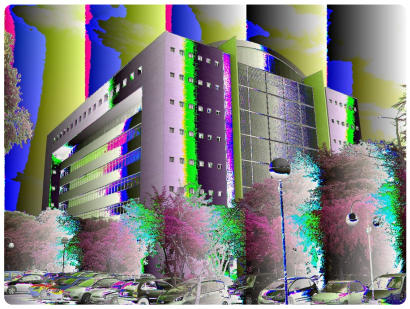
Resultado

O prédio deixa de ser reconhecível. Sobra só a tendência de cor, mais clara em cima por causa do céu e mais escura embaixo. A imagem recuperada também é idêntica à original.

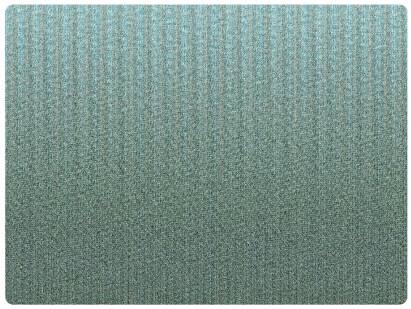
5. Imagens geradas



Original



Confundida



Difundida

6. Como executar

```
make
./bagunceitor predio32.jpg
```

O programa gera `confundida.png`, `recuperada_confusao.png`, `difundida.png` e `recuperada_difusao.png`.